
Software Requirements Specification

for

Event-Driven Ledger

Version 1.0

Prepared by Raj Makwana

Independent Project

24 January 2026

Table of Contents

Table of Contents	2
Revision History	2
1. Introduction	1
1.1 Purpose.....	1
1.2 Intended Audience	1
1.3 Project Scope	1
1.4 References.....	1
2. Overall Description	2
2.1 Project Perspective.....	2
2.2 Project Functions.....	2
2.3 System Actors	2
2.4 Operating Environment	2
2.5 Design and Implementation Constraints.....	2
2.6 Assumptions.....	2
3. External Interface Requirements	3
3.1 Software Interfaces	3
3.2 Communication Interfaces.....	3
4. System Features.....	3
4.1 Wallet Lifecycle management	3
4.1.1 Create	3
4.1.2 Activate	3
4.1.3 Suspend	3
4.1.4 Close	3
4.2 Credit Operation	3
4.3 Debit Operation.....	3
4.4 Read Operation	4
4.5 Reversal Handling.....	4
5. Other Nonfunctional Requirements	4
5.1 Performance	4
5.2 Scalability	4
5.3 Consistency	4
Appendix A: Glossary	4
Appendix B: Analysis Models	4
Appendix C: To Be Determined List.....	4

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This document specifies the software requirements for **Event-Driven Ledger**, a backend system designed to manage wallet-based financial records using an event-driven and ledger-oriented approach. The purpose of this **SRS** is to define the scope, behavior, and constraints of the system at a conceptual and functional level, focusing on correctness, auditability, and reliability of financial state changes. This document describes the system as a standalone ledger component and **does not cover** user-facing applications or external payment processing systems.

1.2 Intended Audience

This document is intended for backend engineers, system designers, architects, and technical reviewers who are involved in the design, implementation, or evaluation of the Event-Driven Ledger system. Readers are expected to have a basic understanding of backend systems and distributed architectures. This SRS provides a high-level view of system requirements and is organized to allow readers to first understand the system's purpose and scope before reviewing detailed functional and non-functional requirements.

1.3 Project Scope

Event-Driven Ledger is a backend financial ledger system responsible for managing wallets and recording all monetary changes as immutable events. **The system supports wallet lifecycle management, financial credit and debit operations, reversal handling, and balance derivation based on historical events.** It is designed to act as a core financial component that can be used by other systems rather than as a complete end-user application. The scope of this system explicitly excludes user authentication, user interfaces, and direct integration with external banking or payment networks, as these concerns are expected to be handled by upstream systems.

1.4 References

PostgreSQL Event Sourcing Github Repository ([Link](#))

2. Overall Description

2.1 Project Perspective

Event-Driven Ledger is a backend financial ledger system designed to act as a core component within a larger service ecosystem. It is not a standalone end-user application, but a foundational ledger engine that other systems use to record and query financial state changes.

2.2 Project Functions

At a high level, the Event-Driven Ledger system provides the following core functions:

- Creation and lifecycle management of wallets, including activation, suspension, and closure.
- Recording of financial credit and debit operations as immutable events.
- Handling of reversal operations to compensate for previously recorded financial actions.
- Derivation of wallet balances and financial state from historical events.
- Support for querying wallet-related information such as balances and transaction history.

2.3 System Actors

The system is accessed by **external and internal system actors** rather than direct human users. These actors represent trusted client systems that interact with the ledger to perform financial operations or retrieve financial data.

2.4 Operating Environment

Event-Driven Ledger operates as a **server-side backend system** deployed within a **distributed computing environment**. It is designed to run as a long-lived service capable of handling **concurrent requests and asynchronous event processing**.

2.5 Design and Implementation Constraints

The design and implementation of the system are constrained by several non-negotiable requirements derived from the wallet domain. All financial and lifecycle changes must be recorded as immutable, append-only events that are never modified or deleted. **Event ordering** must be **preserved** at the wallet level to ensure correct balance derivation. The system must enforce strict domain invariants, such as preventing negative balances and duplicate debits, even in the presence of retries or failures. Consistency for read operations is expected to be eventual rather than immediate, as financial correctness takes precedence over synchronous read accuracy.

2.6 Assumptions

The system assumes that all incoming requests originate from authenticated and authorized upstream systems that provide appropriate contextual information for enforcing operation-level authorization. It also assumes the availability of durable storage for persisting events and reliable communication mechanisms for delivering events to downstream consumers, potentially with at-least-once delivery semantics. The system does not assume global clock synchronization across components and is designed to function correctly without relying on strict time ordering beyond the scope of individual wallets.

3. External Interface Requirements

3.1 Software Interfaces

Event-Driven Ledger interacts with upstream client systems that initiate wallet and financial operations, as well as downstream components that consume financial events for reporting or reconciliation. These interactions are performed through well-defined service interfaces that allow commands to be submitted and financial data to be queried without exposing internal ledger state. The system does not provide any direct user-facing interfaces.

3.2 Communication Interfaces

The system supports both synchronous request–response communication for command submission and queries, and asynchronous message-based communication for event distribution.

4. System Features

4.1 Wallet Lifecycle management

4.1.1 Create

This feature allows the system to create a new wallet with a unique identity. A wallet must be created before any financial or lifecycle operations can be applied to it. Creation establishes the initial state of the wallet and enables future activation and financial activity.

4.1.2 Activate

This feature allows a previously created wallet to transition into an active state in which financial operations are permitted. Only active wallets can receive credits or process debits. Activation confirms that the wallet is ready for financial use.

4.1.3 Suspend

This feature temporarily disables financial activity on a wallet while preserving its existence and historical data. A suspended wallet cannot be credited or debited, but its information and past transactions remain accessible

4.1.4 Close

This feature permanently terminates a wallet and prevents any further lifecycle or financial operations from being applied to it. A closed wallet remains in the system for audit and historical purposes, but it cannot be reactivated or modified in any way.

4.2 Credit Operation

This feature allows funds to be added to an active wallet, increasing its monetary value. Each credit operation is recorded as an immutable financial event.

4.3 Debit Operation

This feature allows funds to be deducted from an active wallet, reducing its monetary value. A debit operation is only permitted if sufficient balance is available and the request has not already been applied.

4.4 Read Operation

This feature allows read-only access to wallet information such as balance, lifecycle status, and transaction history. Query operations do not modify wallet state and are served from derived data representations built from historical events.

4.5 Reversal Handling

This feature allows previously recorded credit or debit operations to be compensated through reversal actions when corrections are required. Reversals do not modify or delete historical data but instead record new events that offset the effects of earlier transactions.

5. Other Nonfunctional Requirements

5.1 Performance

- System shall respond to user requests within **1 seconds under normal load**.
- System shall support **1,000 to 1M concurrent users**.
- API response time shall be **less than 500 milliseconds**.

5.2 Scalability

- System shall handle a **10× increase** in users without downtime.
- System shall scale **horizontally to support increasing traffic**.

5.3 Consistency

- System shall ensure **ACID properties** for transactions.
- Data shall remain consistent across services.

Appendix A: Glossary

This appendix contains definitions of domain terms such as Wallet, Balance, Credit, Debit, Event, and Command, as defined in the **Wallet Domain documentation**.

Appendix B: Analysis Models

This section is intentionally left for future design and modeling artifacts that will be added as the system design evolves.

Appendix C: To Be Determined List

- Exact communication protocols and message formats
- Detailed security and authentication mechanisms
- Monitoring and alerting strategy
- Infrastructure and deployment topology